Automatic Differentiation and Transformer Architecture

Danyang Zhuo

Duke University

Some slides are from CMU's deep learning system course and UCSD's machine learning system course.

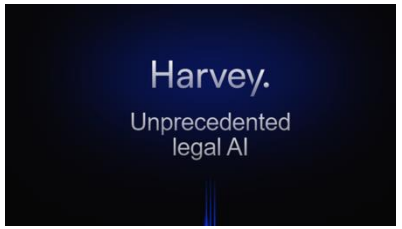
App Layer

Coding

Robotics

OpenEvidence

Medicine



Law

Model Layer

ChatGPT

Gemini

Claude

Infrastructure Layer

Software

SGL

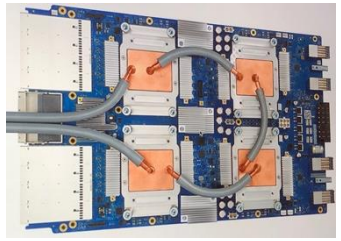
vLLM

JAX

PyTorch

Hardware

GPU



TPU

NVLink

RDMA

App Layer

Cursor
AI Code Editor

Coding

Robotics

OpenEvidence

Medicine

Harvey.
Unprecedented legal AI

Law

Model Layer

ChatGPT

Gemini

Claude

This course

Software

SGL

vLLM

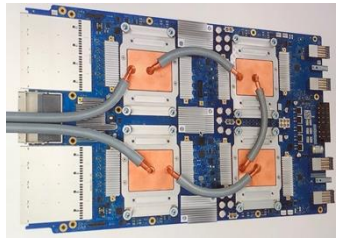
JAX

PyTorch

Infrastructure Layer

Hardware

GPU



TPU

NVLink

RDMA

What’s this course is about?

- Covers the today’s AI software infrastructure stack by surveying recent papers
- Get some feelings about how to do engineering work or research in this field

Course structure

- For each lecture/discussion
  - Students read a paper before the lecture
  - We spend 50% of the lecture time to discuss it together to understand it
  - We spend 50% of the lecture time to learn some background knowledge

- Coding
  - 5 programming assignments. Individual assignments. (50% of grade)
    - AI tools are not allowed for completing programming assignments.
  - 1 research project. Team of 1-3 students. (50% of grade)
    - Using AI tools for coding the research project is allowed.

590 is different from other Duke CS courses

- Not a standard undergrad or graduate-level courses
  - We will not provide support for installing IDEs. We do not teach how to use tools like git and docker. We will not sit with you to help you debug.
  - Content is loosely structured. Papers are not textbooks. They are written assuming readers have certain background.
  - This course is high-paced.

- Please let us know (send Danyang an email)
  - if programming assignments have bugs
  - if certain topic is not clear
  - if we are wrong (think about each lecture as a discussion rather than a lecture)

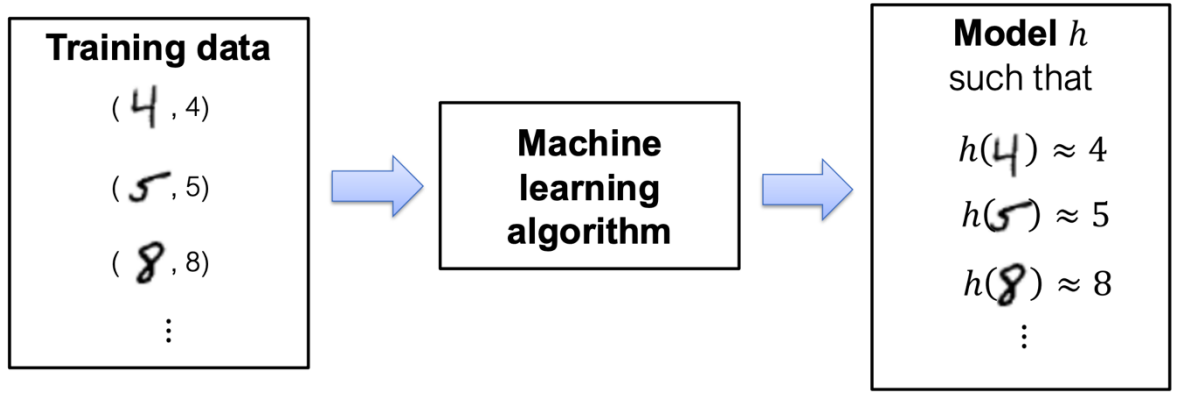
Some tips

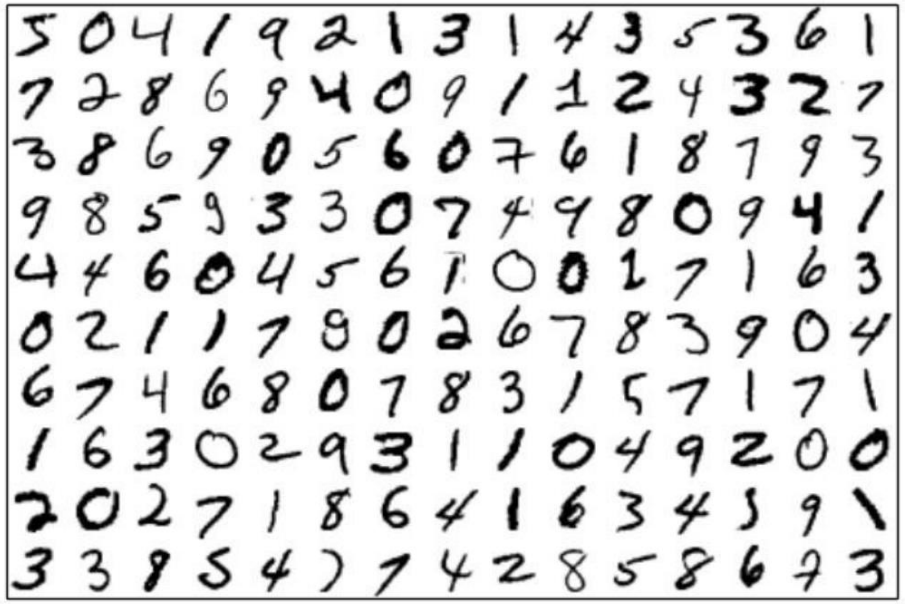
- Read papers carefully. Always think about how things are realized in code.

- Start programming assignments early.
  - These are **harder** than the standard programming assignments in your undergrad- and grad-level courses.

- Book some time each week with your project team to make progress consistently.

Deep learning





MNIST Dataset

- Training data: $x^{(i)} \in \mathbb{R}^n, y^{(i)} \in \{1, \dots, k\}$ for $i = 1, \dots m$
- n = dimensionality of the input data
- k = number of different classes / labels
- m = number of points in the training set

Example: classification of 28x28 MNIST digits

- $n = 28 \cdot 28 = 784$
- $k = 10$
- $m = 60{,}000$

Two-layer neural network

$$\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1 \qquad \text{(Input to first layer)}$$
$$\mathbf{a}_1 = \sigma(\mathbf{z}_1) \qquad \text{(Activation of first layer)}$$
$$\mathbf{z}_2 = \mathbf{W}_2\mathbf{a}_1 + \mathbf{b}_2 \qquad \text{(Input to second layer)}$$
$$\hat{\mathbf{y}} = f(\mathbf{z}_2) \qquad \text{(Output layer)}$$

- $\mathbf{x}$: Input features.
- $\mathbf{W}_1, \mathbf{W}_2$: Weight matrices.
- $\mathbf{b}_1, \mathbf{b}_2$: Bias vectors.
- σ: Activation function (e.g., ReLU or sigmoid).
- f: Output activation function (e.g., softmax for classification).

- $\mathbf{x} \in \mathbb{R}^{784\times 60000}$. Each pixel is a feature.
- $\mathbf{W}_1 \in \mathbb{R}^{800\times 784}$, $\mathbf{b}_1 \in \mathbb{R}^{800\times 1}$
- $\mathbf{W}_2 \in \mathbb{R}^{10\times 800}$, $\mathbf{b}_2 \in \mathbb{R}^{10\times 1}$
- σ is ReLU
- f is Softmax
- L is cross-entropy.

$$\ell_{ce}(h(x), y) = -\log p(\text{label} = y)$$

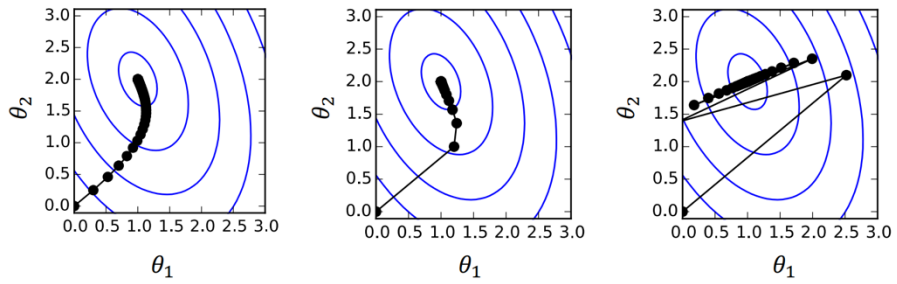
Gradient descend

$$\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1 \qquad \text{(Input to first layer)}$$
$$\mathbf{a}_1 = \sigma(\mathbf{z}_1) \qquad \text{(Activation of first layer)}$$
$$\mathbf{z}_2 = \mathbf{W}_2\mathbf{a}_1 + \mathbf{b}_2 \qquad \text{(Input to second layer)}$$
$$\hat{\mathbf{y}} = f(\mathbf{z}_2) \qquad \text{(Output layer)}$$

1. **Forward Pass**: Compute the output $\hat{\mathbf{y}}$ and the loss L.
2. **Backward Pass**: Use the chain rule to compute gradients with respect to $\mathbf{W}_1$, $\mathbf{b}_1$, $\mathbf{W}_1$, $\mathbf{b}_2$:
   - $\frac{\partial L}{\partial \mathbf{W}_2} = \frac{\partial L}{\partial \mathbf{z}_2}\frac{\partial \mathbf{z}_2}{\partial \mathbf{W}_2}$
   - $\frac{\partial L}{\partial \mathbf{b}_2} = \frac{\partial L}{\partial \mathbf{z}_2}\frac{\partial \mathbf{z}_2}{\partial \mathbf{b}_2}$
   - $\frac{\partial L}{\partial \mathbf{z}_1} = \frac{\partial L}{\partial \mathbf{z}_2}\frac{\partial \mathbf{z}_2}{\partial \mathbf{a}_1}\frac{\partial \mathbf{a}_1}{\partial \mathbf{z}_1}$
   - $\frac{\partial L}{\partial \mathbf{W}_1} = \frac{\partial L}{\partial \mathbf{z}_1}\frac{\partial \mathbf{z}_1}{\partial \mathbf{W}_1}$
   - $\frac{\partial L}{\partial \mathbf{b}_1} = \frac{\partial L}{\partial \mathbf{z}_1}\frac{\partial \mathbf{z}_1}{\partial \mathbf{b}_1}$

$$\theta \coloneqq \theta - \alpha \nabla_\theta f(\theta)$$

where $\alpha > 0$ is a *step size* or *learning rate*



$\alpha = 0.05$ $\alpha = 0.2$ $\alpha = 0.42$

Stochastic gradient descend (SGD)

Sample a minibatch of data $X \in \mathbb{R}^{B \times n}, y \in \{1, \dots, k\}^B$

Update parameters $\theta := \theta - \frac{\alpha}{B} \sum_{i=1}^{B} \nabla_\theta \ell\left(h_\theta\left(x^{(i)}\right), y^{(i)}\right)$

Training loop in NumPy

```
start = i * batch_size
end = start + batch_size
X_batch = X_train[start:end]
y_batch = y_train[start:end]

z1 = np.dot(X_batch, W1) + b1
a1 = relu(z1)
z2 = np.dot(a1, W2) + b2
predictions = softmax(z2)

loss = cross_entropy_loss(predictions, y_batch)
epoch_loss += loss

dz2 = predictions - y_batch
dW2 = np.dot(a1.T, dz2) / batch_size
db2 = np.sum(dz2, axis=0, keepdims=True) / batch_size

da1 = np.dot(dz2, W2.T)
dz1 = da1 * relu_derivative(z1)
dW1 = np.dot(X_batch.T, dz1) / batch_size
db1 = np.sum(dz1, axis=0, keepdims=True) / batch_size

W2 -= learning_rate * dW2
b2 -= learning_rate * db2
W1 -= learning_rate * dW1
b1 -= learning_rate * db1
```

Hand-written gradient calculation

Hand-written optimizer

Training loop with autodiff

```
start = i * batch_size
end = start + batch_size
X_batch = X_train[start:end]
y_batch = y_train[start:end]

z1 = np.dot(X_batch, W1) + b1
a1 = relu(z1)
z2 = np.dot(a1, W2) + b2
predictions = softmax(z2)

loss = cross_entropy_loss(predictions, y_batch)
epoch_loss += loss

dz2 = predictions - y_batch
dW2 = np.dot(a1.T, dz2) / batch_size
db2 = np.sum(dz2, axis=0, keepdims=True) / batch_size

da1 = np.dot(dz2, W2.T)
dz1 = da1 * relu_derivative(z1)
dW1 = np.dot(X_batch.T, dz1) / batch_size
db1 = np.sum(dz1, axis=0, keepdims=True) / batch_size

W2 -= learning_rate * dW2
b2 -= learning_rate * db2
W1 -= learning_rate * dW1
b1 -= learning_rate * db1
```

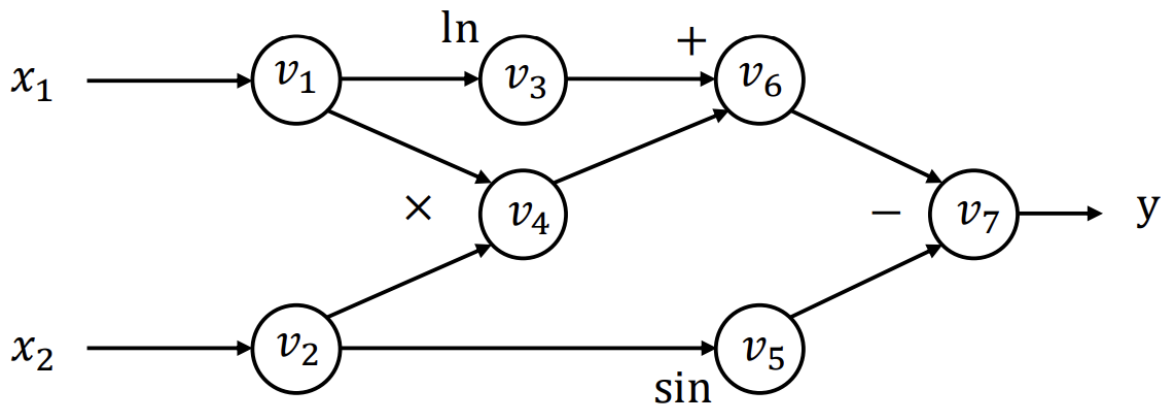
Numpy

```
start = i * batch_size
end = start + batch_size
X_batch = X_train[start:end]
y_batch = torch.argmax(y_train[start:end], dim=1)
outputs = model(X_batch)
loss = criterion(outputs, y_batch)
epoch_loss += loss.item()

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

PyTorch

Automatic differentiation

$$y = f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin x_2$$



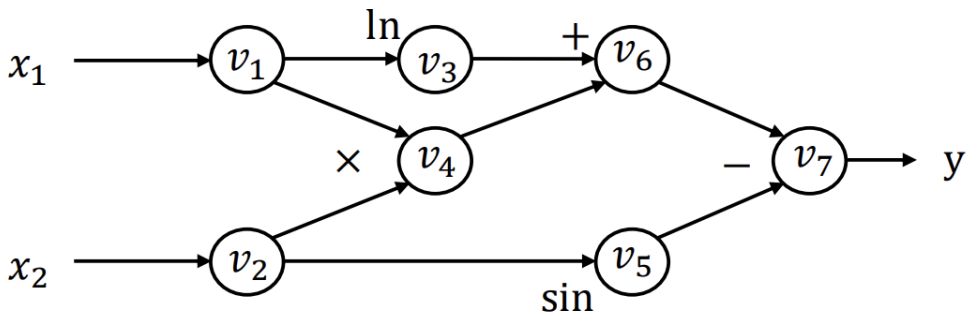
Forward evaluation trace

$$v_1 = x_1 = 2$$
$$v_2 = x_2 = 5$$
$$v_3 = \ln v_1 = \ln 2 = 0.693$$
$$v_4 = v_1 \times v_2 = 10$$
$$v_5 = \sin v_2 = \sin 5 = -0.959$$
$$v_6 = v_3 + v_4 = 10.693$$
$$v_7 = v_6 - v_5 = 10.693 + 0.959 = 11.652$$
$$y = v_7 = 11.652$$

Each node represent an (intermediate) value in the computation. Edges present input output relations.

Forward-mode autodiff

$$y = f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin x_2$$



Define $\dot{v}_i = \frac{\partial v_i}{\partial x_1}$

We can then compute the $\dot{v}_i$ iteratively in the forward topological order of the computational graph

Forward evaluation trace

$$v_1 = x_1 = 2$$
$$v_2 = x_2 = 5$$
$$v_3 = \ln v_1 = \ln 2 = 0.693$$
$$v_4 = v_1 \times v_2 = 10$$
$$v_5 = \sin v_2 = \sin 5 = -0.959$$
$$v_6 = v_3 + v_4 = 10.693$$
$$v_7 = v_6 - v_5 = 10.693 + 0.959 = 11.652$$
$$y = v_7 = 11.652$$

Forward AD trace

$$\dot{v}_1 = 1$$
$$\dot{v}_2 = 0$$
$$\dot{v}_3 = \dot{v}_1 / v_1 = 0.5$$
$$\dot{v}_4 = \dot{v}_1 v_2 + \dot{v}_2 v_1 = 1 \times 5 + 0 \times 2 = 5$$
$$\dot{v}_5 = \dot{v}_2 \cos v_2 = 0 \times \cos 5 = 0$$
$$\dot{v}_6 = \dot{v}_3 + \dot{v}_4 = 0.5 + 5 = 5.5$$
$$\dot{v}_7 = \dot{v}_6 - \dot{v}_5 = 5.5 - 0 = 5.5$$

Now we have $\frac{\partial y}{\partial x_1} = \dot{v}_7 = 5.5$

Problems with forward-mode autodiff

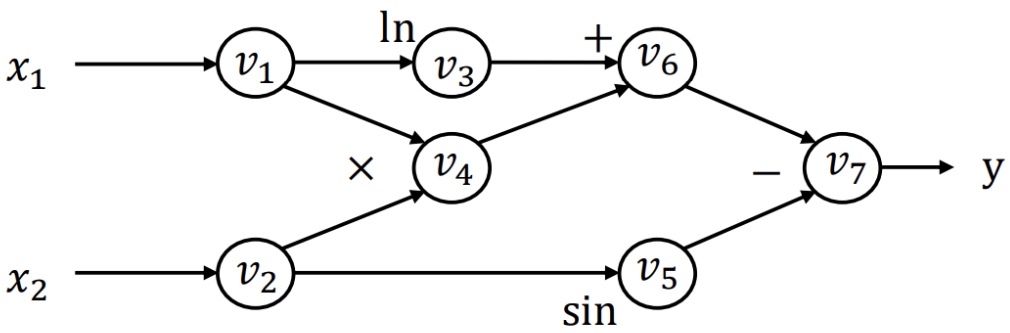
For $f: \mathbb{R}^n \to \mathbb{R}^k$, we need n forward AD passes to get the gradient with respect to each input.

We mostly care about the cases where $k = 1$ and large n .

In order to resolve the problem efficiently, we need to use another kind of AD.

Reverse-mode autodiff

$$y = f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin x_2$$



Forward evaluation trace

$$v_1 = x_1 = 2$$
$$v_2 = x_2 = 5$$
$$v_3 = \ln v_1 = \ln 2 = 0.693$$
$$v_4 = v_1 \times v_2 = 10$$
$$v_5 = \sin v_2 = \sin 5 = -0.959$$
$$v_6 = v_3 + v_4 = 10.693$$
$$v_7 = v_6 - v_5 = 10.693 + 0.959 = 11.652$$
$$y = v_7 = 11.652$$

Define adjoint $\overline{v_i} = \frac{\partial y}{\partial v_i}$

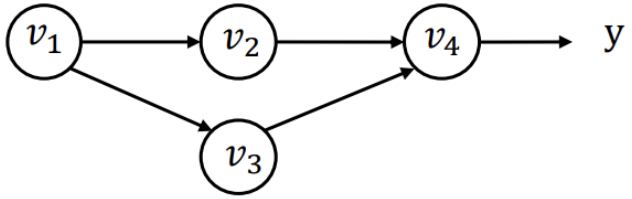
We can then compute the $\overline{v_i}$ iteratively in the **reverse** topological order of the computational graph

Reverse AD evaluation trace

$$\overline{v_7} = \frac{\partial y}{\partial v_7} = 1$$
$$\overline{v_6} = \overline{v_7}\frac{\partial v_7}{\partial v_6} = \overline{v_7} \times 1 = 1$$
$$\overline{v_5} = \overline{v_7}\frac{\partial v_7}{\partial v_5} = \overline{v_7} \times (-1) = -1$$
$$\overline{v_4} = \overline{v_6}\frac{\partial v_6}{\partial v_4} = \overline{v_6} \times 1 = 1$$
$$\overline{v_3} = \overline{v_6}\frac{\partial v_6}{\partial v_3} = \overline{v_6} \times 1 = 1$$
$$\overline{v_2} = \overline{v_5}\frac{\partial v_5}{\partial v_2} + \overline{v_4}\frac{\partial v_4}{\partial v_2} = \overline{v_5} \times \cos v_2 + \overline{v_4} \times v_1 = -0.284 + 2 = 1.716$$
$$\overline{v_1} = \overline{v_4}\frac{\partial v_4}{\partial v_1} + \overline{v_3}\frac{\partial v_3}{\partial v_1} = \overline{v_4} \times v_2 + \overline{v_3}\frac{1}{v_1} = 5 + \frac{1}{2} = 5.5$$

Autodiff

v_1 is being used in multiple pathways (v_2 and v_3)



y can be written in the form of $\mathrm{y} = \mathrm{f}(\mathrm{v}_2, \mathrm{v}_3)$

$$\overline{v_1} = \frac{\partial y}{\partial v_1} = \frac{\partial f(v_2,v_3)}{\partial v_2}\frac{\partial v_2}{\partial v_1} + \frac{\partial f(v_2,v_3)}{\partial v_3}\frac{\partial v_3}{\partial v_1} = \overline{v_2}\frac{\partial v_2}{\partial v_1} + \overline{v_3}\frac{\partial v_3}{\partial v_1}$$

Define partial adjoint $\overline{v_{i\to j}} = \overline{v_j}\dfrac{\partial v_j}{\partial v_i}$ for each input output node pair i and j

$$\overline{v_i} = \sum_{j \in next(i)} \overline{v_{i\to j}}$$

We can compute partial adjoints separately then sum them together

Autodiff

```
def gradient(out):
    node_to_grad = {out: [1]}

    for i in reverse_topo_order(out):
        v̄_i = Σ_j v̄_{i→j} = sum(node_to_grad[i])

        for k ∈ inputs(i):
            compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
            append v̄_{k→i} to node_to_grad[k]

    return adjoint of input v̄_input
```

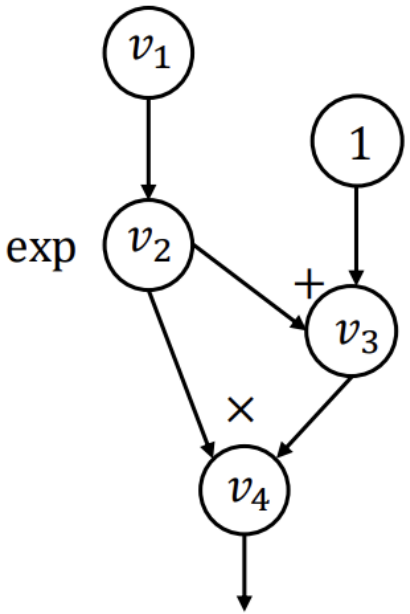
Dictionary that records a list of partial adjoints of each node

Sum up partial adjoints

"Propagates" partial adjoint to its input

Autodiff

```
def gradient(out):
  node_to_grad = {out:  [1]}
  for i in reverse_topo_order(out):
    v̄_i = Σ_j v̄_{i→j} = sum(node_to_grad[i])
    for k ∈ inputs(i):
        compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
        append v̄_{k→i} to node_to_grad[k]
    return adjoint of input v̄_input
```

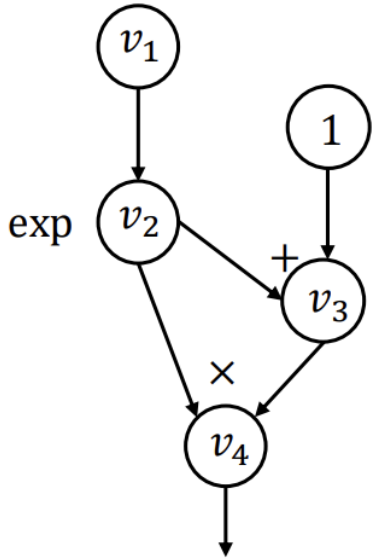


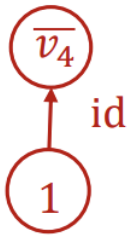
Autodiff

```
def gradient(out):
  node_to_grad = {out:  [1]}
  for i in reverse_topo_order(out):
    v̄_i = Σ_j v̄_{i→j} = sum(node_to_grad[i])
    for k ∈ inputs(i):
        compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
        append v̄_{k→i} to node_to_grad[k]
    return adjoint of input v̄_input
```

$i = 4$

```
node_to_grad: {
  4: [v̄_4]
}
```





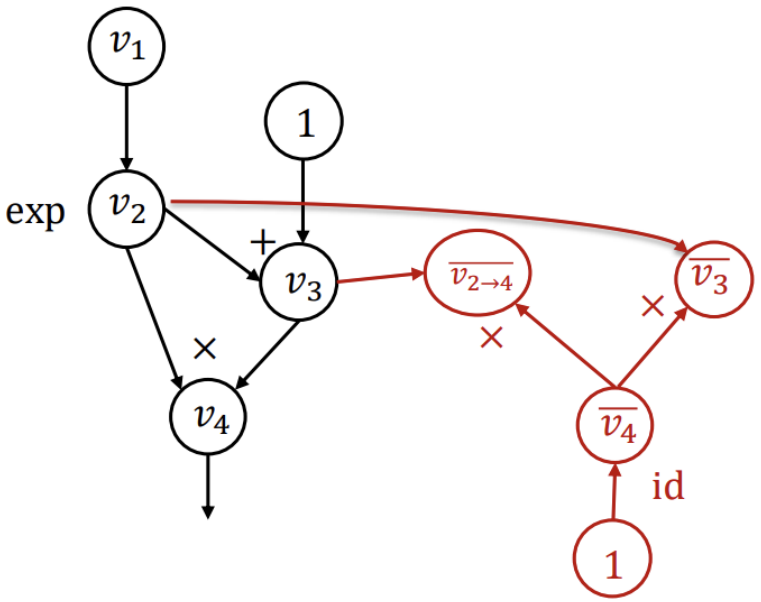
NOTE: id is identity function

Autodiff

```
def gradient(out):
  node_to_grad = {out:  [1]}
  for i in reverse_topo_order(out):
    v̄_i = Σ_j v̄_{i→j} = sum(node_to_grad[i])
    for k ∈ inputs(i):
        compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
        append v̄_{k→i} to node_to_grad[k]
    return adjoint of input v̄_input
```

$i = 4$

```
node_to_grad: {
  2: [v̄_{2→4}]
  3: [v̄_3]
  4: [v̄_4]
}
```



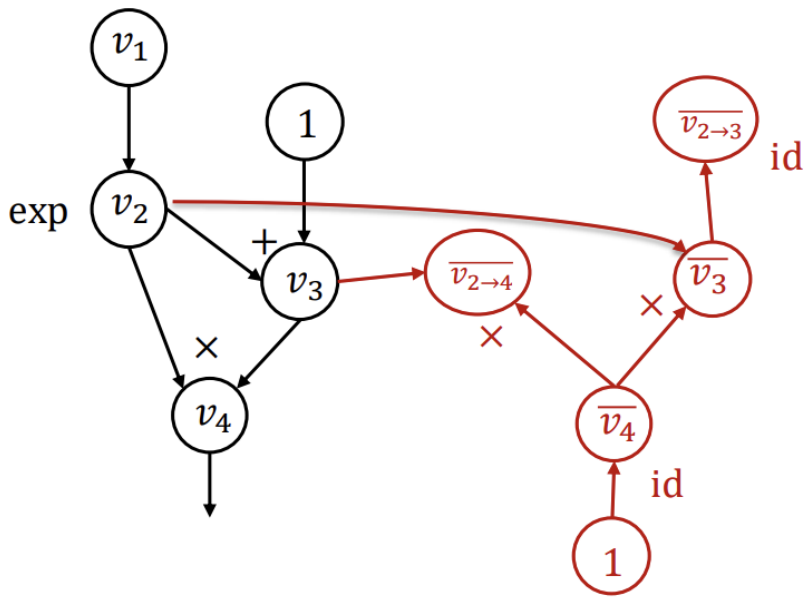
NOTE: id is identity function

Autodiff

```
def gradient(out):
  node_to_grad = {out:  [1]}
  for i in reverse_topo_order(out):
    v̄_i = Σ_j v̄_{i→j} = sum(node_to_grad[i])
    for k ∈ inputs(i):
        compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
        append v̄_{k→i} to node_to_grad[k]
    return adjoint of input v̄_input
```

$i = 3$

```
node_to_grad: {
  2: [v̄_{2→4}, v̄_{2→3}]
  3: [v̄_3]
  4: [v̄_4]
}
```



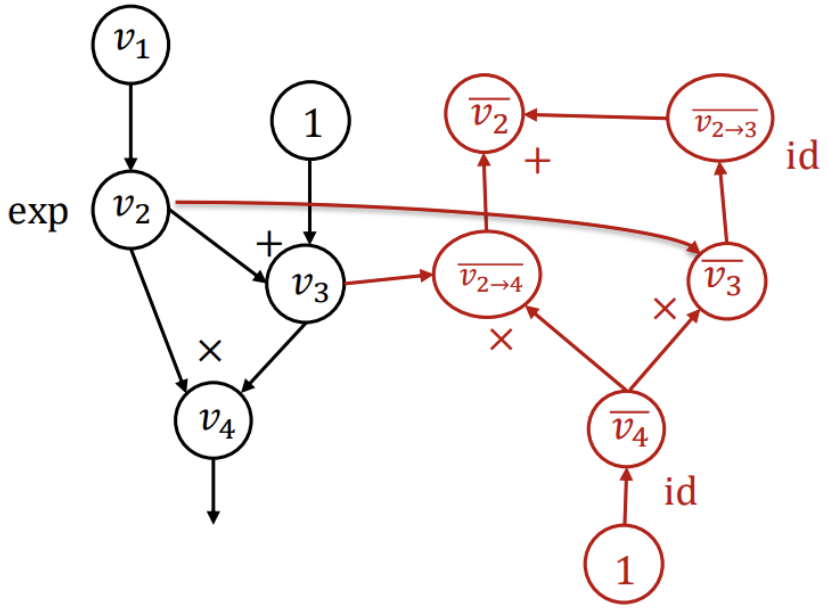
NOTE: id is identity function

Autodiff

```
def gradient(out):
  node_to_grad = {out:  [1]}
  for i in reverse_topo_order(out):
    v̄_i = Σ_j v̄_{i→j} = sum(node_to_grad[i])
    for k ∈ inputs(i):
        compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
        append v̄_{k→i} to node_to_grad[k]
    return adjoint of input v̄_input
```

$i = 2$

```
node_to_grad: {
  2: [v̄_{2→4}, v̄_{2→3}]
  3: [v̄_3]
  4: [v̄_4]
}
```



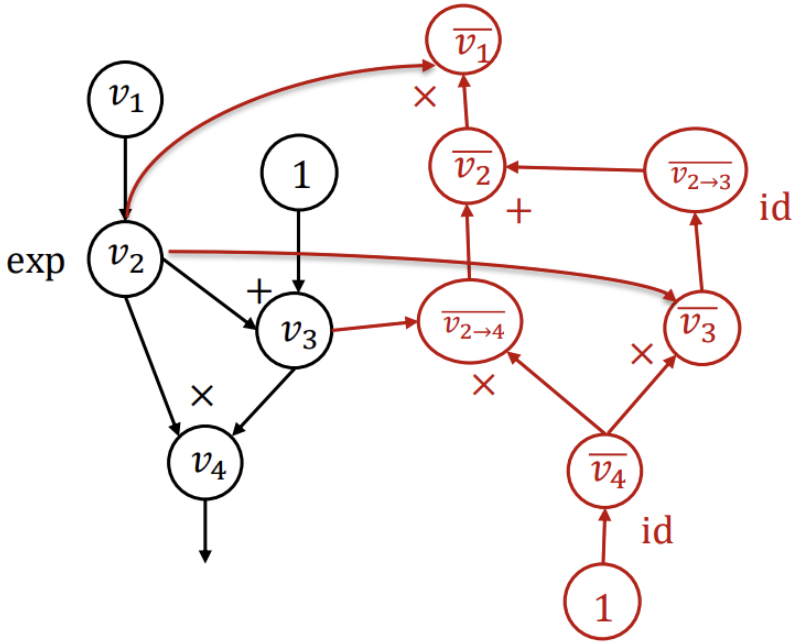
NOTE: id is identity function

Autodiff

```
def gradient(out):
  node_to_grad = {out:  [1]}
  for i in reverse_topo_order(out):
    v̄_i = Σ_j v̄_{i→j} = sum(node_to_grad[i])
    for k ∈ inputs(i):
        compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
        append v̄_{k→i} to node_to_grad[k]
    return adjoint of input v̄_input
```

$i = 2$

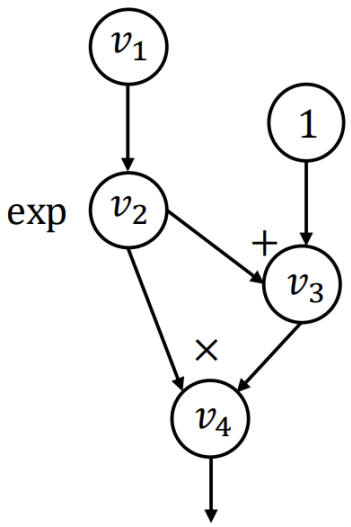
```
node_to_grad: {
  1: [v̄_1]
  2: [v̄_{2→4}, v̄_{2→3}]
  3: [v̄_3]
  4: [v̄_4]
}
```



NOTE: id is identity function

Declarative vs. imperative

Example framework: Tensorflow 1.0



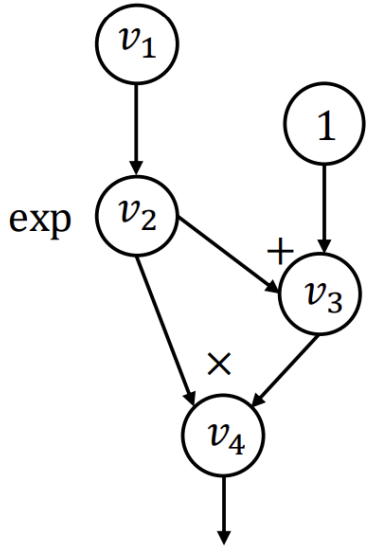

```
import tensorflow as tf

v1 = tf.Variable()
v2 = tf.exp(v1)
v3 = v2 + 1
v4 = v2 * v3

sess = tf.Session()
value4 = sess.run(v4, feed_dict={v1: numpy.array([1]})
```

First declare the computational graph

Then execute the graph by feeding input value

Declarative vs. imperative

Example framework: PyTorch (needle:)




```
import needle as ndl

v1 = ndl.Tensor([1])
v2 = ndl.exp(v1)
v3 = v2 + 1
v4 = v2 * v3
```

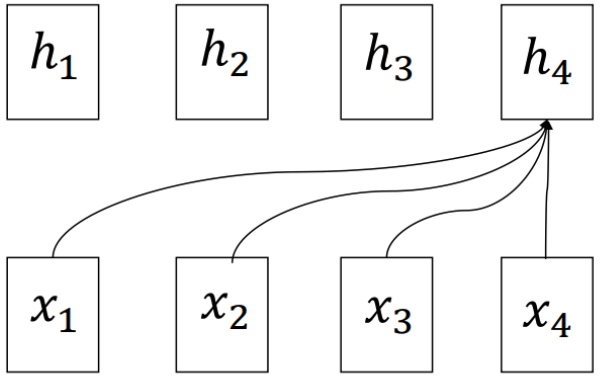
Executes computation as we construct the computational graph
Allow easy mixing of python control flow and construction

```
if v4.numpy() > 0.5:
    v5 = v4 * 2
else:
    v5 = v4
v5.backward()
```

“Attention” Mechanism

Generally refers to the approach that weighted combine individual states

Attention output



Hidden states from previous layer

$$h_t = \sum_{i=1}^{t} s_i x_t$$

Intuitively s_i is “attention score” that computes how relevant the position i’s input is to this current hidden output

There are different methods to decide how attention score is being computed

Self-Attention Operation

Self attention refers to a particular form of attention mechanism.

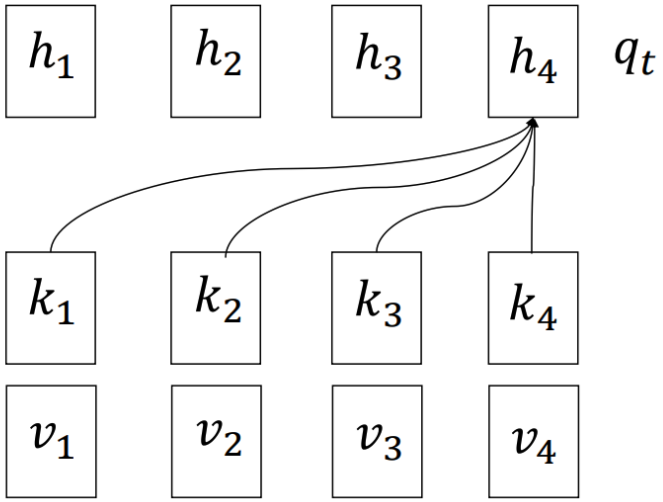
Given three inputs $Q, K, V \in \mathbb{R}^{T \times d}$ (“queries”, “keys”, “values”)

Define the self-attention as:

$$\mathrm{SelfAttention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^T}{d^{1/2}}\right)V$$

A Closer Look at Self-Attention

Use q_t, k_t, v_t to refers to row t of the K matrix



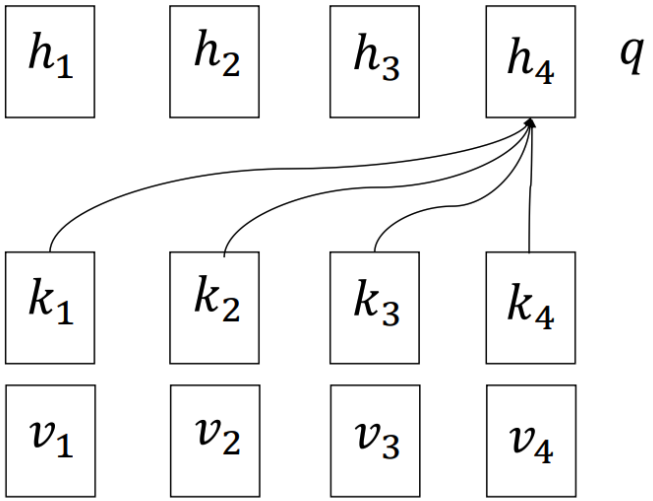
Ask the following question:

How to compute the output h_t, based on q_t, K, V one timestep t

To keep presentation simple, we will drop suffix t and just use q to refer to q_t in next few slide

A Closer Look at Self-Attention

Use q_t, k_t, v_t to refers to row t of the K matrix



Conceptually, we compute the output in the following two steps:

Pre-softmax "attention score"

$$s_i = \frac{1}{\sqrt{d}} q k_i^T$$

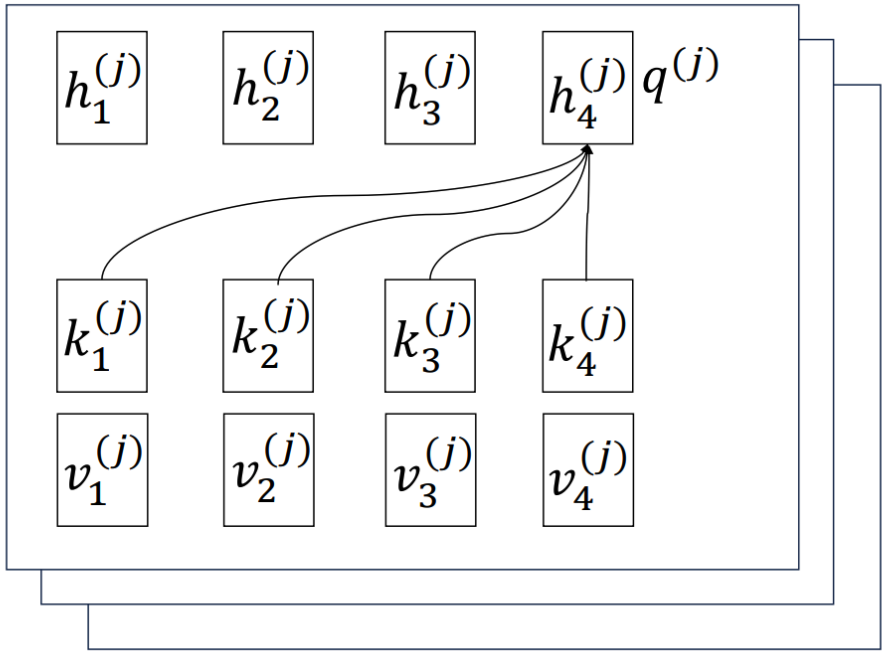
Weighed average via softmax

$$h = \sum_i \mathrm{softmax}(s)_i v_i = \frac{\sum_i \exp(s_i) v_i}{\sum_j \exp(s_j)}$$

Intuition: s_i computes the relevance of k_i to the query q,
then we do weighted sum of values proportional to their relevance

Multi-Head Attention

Have multiple "attention heads" $Q^{(j)}, K^{(j)}, V^{(j)}$ denotes j-th attention head



Apply self-attention in each attention head

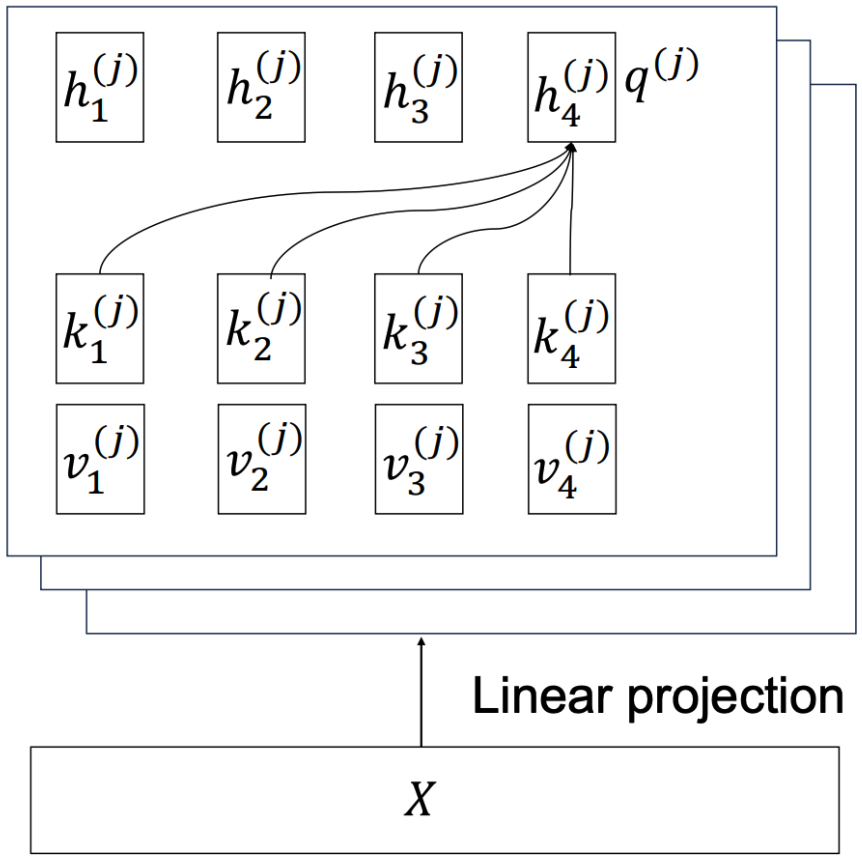
$$\mathrm{SelfAttention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^T}{d^{1/2}}\right)V$$

Concatenate all output heads together as output

Each head can correspond to different kind of information.
Sometimes we can share the heads: GQA(group query attention) all heads share K, V but have different Q

How to get Q K V?

Obtain Q, K, V from previous layer's hidden state X by linear projection



$$Q = XW_q$$
$$K = XW_K$$
$$V = XW_V$$

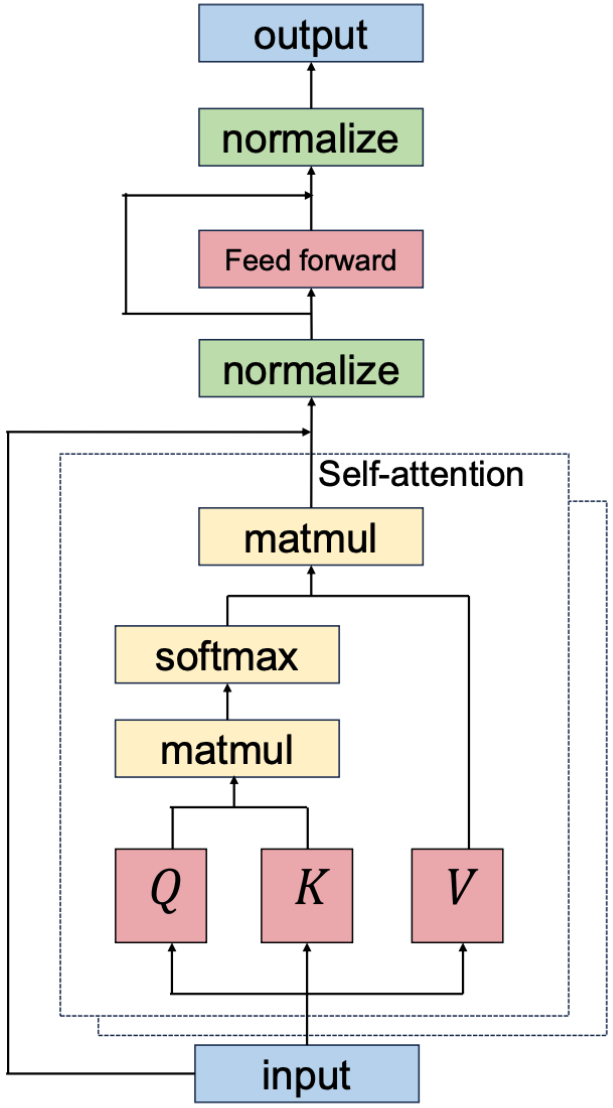
Can compute all heads and Q, K, V together then split/reshape out into individual Q, K, V with multiple heads

Transformer Block

A typical transformer block

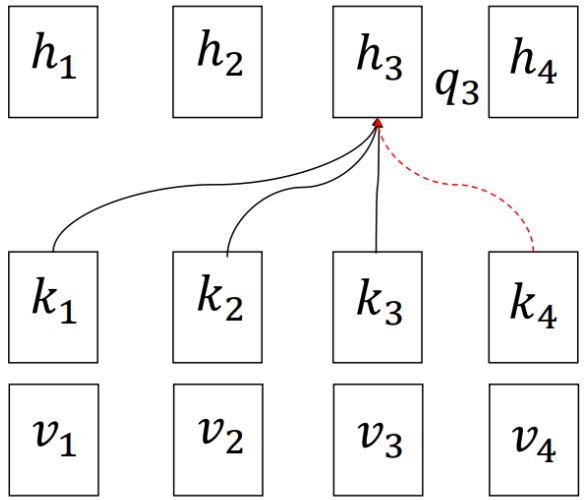
$$Z = \text{SelfAttention}(XW_K, XW_Q, XW_V)$$
$$Z = \text{LayerNorm}(X + Z)$$
$$H = \text{LayerNorm}(\text{ReLU}(ZW_1)W_2 + Z)$$

(multi-head) self-attention, followed by a linear layer and ReLU and some additional residual connections and normalization



Masked Self-Attention

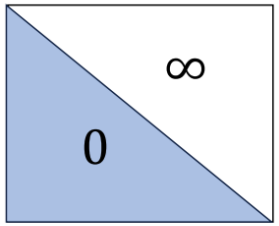
In the matrix form, we are computing weighted average over all inputs



In auto regressive models, usually it is good to maintain casual relation, and only attend to some of the inputs (e.g. skip the red dashed edge on the left). We can add "attention mask"

$$\text{MaskedSelfAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{d^{1/2}} - M\right)V$$

$$M_{ij} = \begin{cases} \infty, j > i \\ 0, j \leq i \end{cases}$$



Only attend to previous inputs. Depending on input structure and model, attention mask can change.

We can also simply skip the computation that are masked out if there is a special implementation to do so